DA3408 Assignment 3

Eshika Nahata (DA24B004)

1. Summary

The same cleaning pipeline (ingest, cleanse, join, UDF-derived speed feature, Parquet export) was implemented in PySpark and Ray Data and run on a two-worker cluster for each framework, on one laptop. The outputs are identical (87,333,714 rows, matching content hash). In this run Spark finished in 220.19 s and Ray in 1816.4 s, so Spark was 8.25x faster end to end. Ray had the smaller Python-UDF overhead (0.84 s vs 6.47 s) and a lower peak system memory (11.97 GB vs 13.17 GB). All numbers come from a single run per framework.

2. Setup

2.1 Dataset

Five NYC TLC Parquet files plus the zone lookup CSV, 2.0 GB on disk in total (du -sh data): four High-Volume FHV files (fhvhv_tripdata_2026-01 to 2026-04), one Yellow Taxi file (yellow_tripdata_2026-01) and taxi_zone_lookup.csv. Raw input is 87,595,259 rows. The two trip schemas are mapped to one common schema (pickup/dropoff time, pickup/dropoff location ID, trip miles, fare) and tagged with a source column.

2.2 Hardware and software

Item	Value
Machine	Single laptop, AMD Ryzen 7 5700U, 12 GiB RAM (free -h), 4 GiB swap
Software	Python 3.14.4, PySpark 4.2.0, Ray 2.58.0
Spark cluster	Standalone master spark://127.0.0.1:7077; 2 workers, 2 cores and 4.0 GiB each; one application using 4 cores and 3.0 GiB per executor
Ray cluster	ray.cluster_utils.Cluster on the same machine: head node with 0 CPUs plus 2 worker nodes with 2 CPUs each (object store set to 2 GB per worker); Ray Data used for all processing

2.3 Cluster evidence

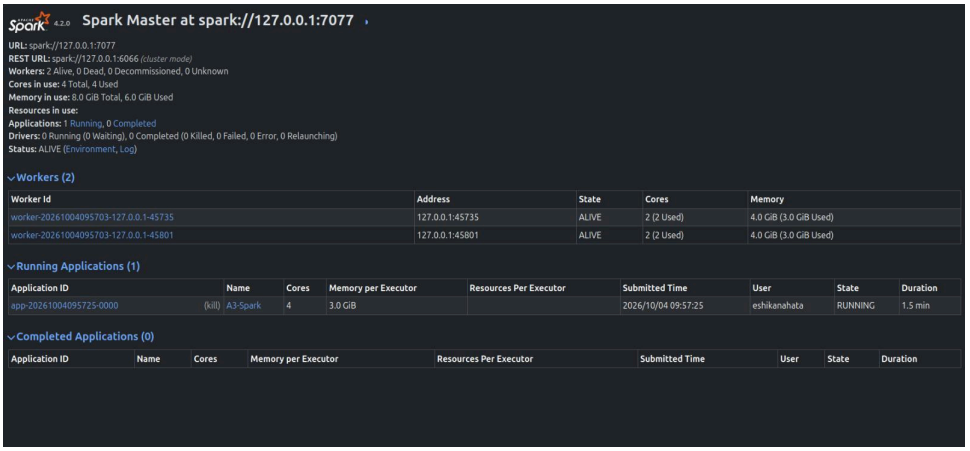


Figure 1. Spark Master UI: 2 workers alive, 4 of 4 cores in use, application A3-Spark running.

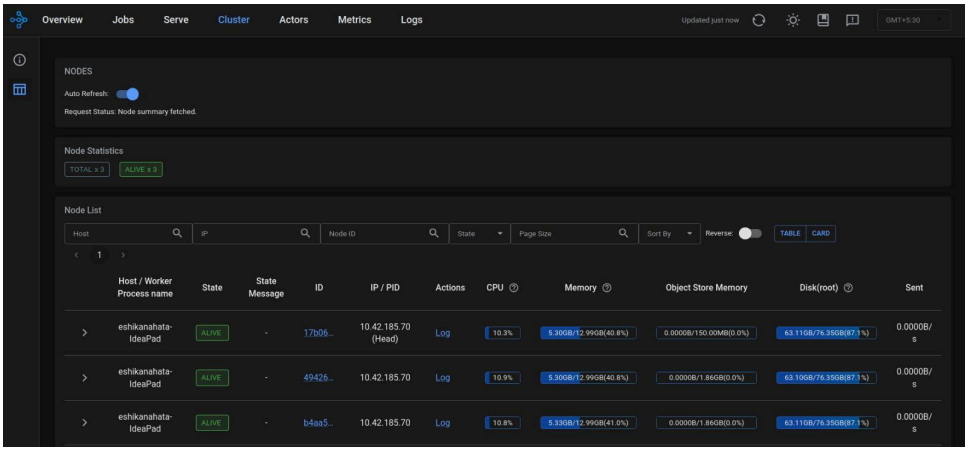


Figure 2. Ray Dashboard, Cluster view: 3 alive nodes (the head plus the 2 workers).

3. Pipeline and Parity

Both implementations follow the same logic (spark_clean.py, ray_clean.py, with shared constants and the speed function in common.py):

- **Ingest:** read the Parquet files, select and rename the common columns, cast types (timestamps, integer IDs, double miles and fare).
- **Cleanse:** drop rows with a null in any key column, drop exact duplicates, derive trip_time_sec, keep only rows with trip_miles > 0, trip_time_sec > 0 and fare >= 0.
- **Transform:** two inner joins to the zone lookup (pickup and drop-off zone and borough); compute speed_mph with a Python UDF (miles / hours, rounded to 2 decimals, null if outside (0, 100] mph) and drop null speeds; add pickup_hour, pickup_dow and is_weekend.
- **Export:** write the 16-column result to Parquet.

Where the engines differ in implementation (not in logic): Spark runs the whole job as one query, using dropDuplicates() and a pandas_udf (Arrow). Ray runs the data in 4 hash-bucket passes (--passes 4, the default), each of which re-reads the inputs and processes one bucket to keep memory low; duplicates are removed with a row hash and a hash-partitioned shuffle, and the same Python function is applied through map_batches.

Parity check. verify_parity.py computes the row count, a sum of per-row xxhash64 over all 16 columns and the sum of speed_mph for each output:

	Rows	Hash sum	sum(speed_mph)
Spark	87,333,714	-47985407622463961372187	1136114430.0
Ray	87,333,714	-47985407622463961372187	1136114430.0

Result: **PARITY: PASS** (identical rows and content). Of the 87,595,259 raw rows, 261,545 (0.30%) were removed by the pipeline in both engines.

4. Performance Results

4.1 Execution time

Wall-clock is measured from the start of ingestion to the end of the Parquet write (time.perf_counter), excluding session or cluster start-up and the final verification count.

Metric	Spark	Ray	Ratio
Wall-clock (s)	220.19	1816.4	Ray 8.25x slower
Wall-clock (min)	3.67	30.27	
Raw rows / s	397,817	48,225	8.25x

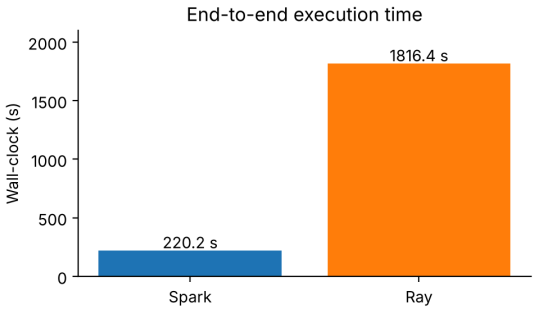


Figure 3. End-to-end execution time.

4.2 Resource utilization

Monitored with monitor.py (psutil), sampling once per second. CPU and memory are system-wide for the whole laptop, not per process, and the idle baseline was measured for 3 s before each run. Spark and Ray both had 4 cores assigned.

Metric	Spark	Ray
Peak CPU (%)	86.8	67.2
Average CPU (%)	39.6	12.2
Peak memory used (GB)	13.17	11.97
Idle baseline memory (GB)	4.42	4.71
Peak memory above baseline (GB)	8.75	7.26
Samples (1 s each)	233	1837

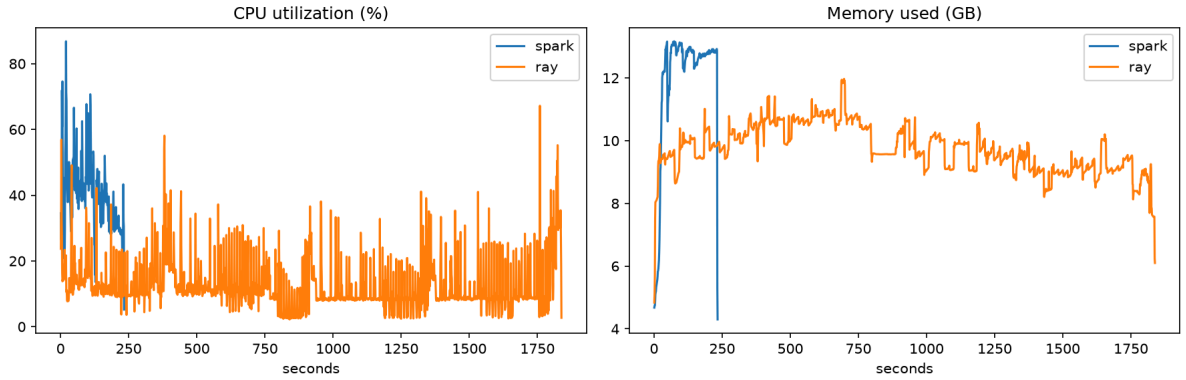


Figure 4. System-wide CPU and memory over time for both runs.

Spark drives the CPU harder and for a short time: a higher peak (86.8% vs 67.2%) and an average more than three times Ray's (39.6% vs 12.2%), with memory rising within about the first minute and staying near 13 GB until the job ends. Ray's CPU trace stays mostly low with frequent dips for the whole 30 minutes, and its memory sits around 9 to 11 GB, drifting down across the passes. Ray therefore used less memory at peak, but over a run about 8 times longer.

5. Python UDF Deep-Dive

udf_benchmark.py isolates the speed_mph computation on the cleaned output (87,333,714 rows) and times each variant on the corresponding cluster, taking the median of repeated timings after a warm-up. Spark results are written with the noop sink, so no Parquet output is included.

Variant	Time (s)	Overhead (s)
Spark baseline (trivial JVM op)	1.78	
Spark native JVM expression	7.23	
Spark pandas_udf (Arrow)	13.70	6.47 vs native JVM
Ray identity map_batches	4.29	
Ray Python function map_batches	5.13	0.84 vs identity

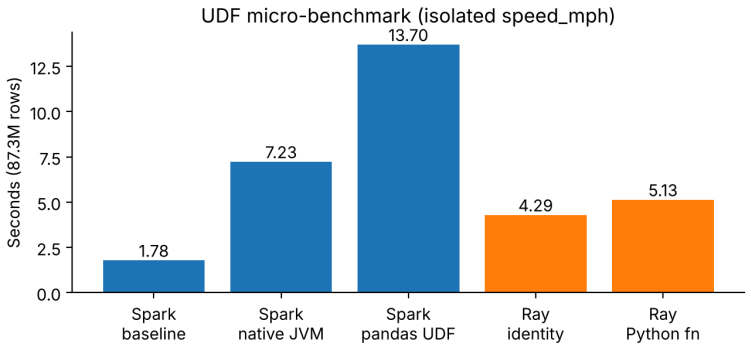


Figure 5. UDF micro-benchmark times.

Observations. Running the speed calculation as a pandas UDF roughly doubled Spark's time compared with the same calculation as a native JVM expression (13.70 s vs 7.23 s, +89%); the extra 6.47 s is the cost of moving batches from the JVM to a Python worker and back. In Ray, the same Python function added 0.84 s (+20%) over an identity map_batches, because the data is already handled as pandas batches inside Python worker processes and no language boundary is crossed. That is the Python-native advantage: the incremental cost of custom Python logic was about 7.7 times smaller in Ray than in Spark in this test.

Caveats. The two overheads are not strictly like-for-like: Spark's is measured against a native JVM version of the same maths, while Ray has no JVM alternative and is measured against an identity function that still includes reading and batching in Python. The absolute times also include different scan and scheduling paths. Finally, the UDF numbers (at most 13.70 s) are small next to the full-pipeline times (220 s and 1816 s), so the UDF is not what separates the two engines' total time.

6. Discussion and Winner

Winner for this pipeline and setup: Apache Spark. It finished 8.25x faster and used the CPU much more fully, with identical output. Ray's advantages in this run were the lower Python-UDF overhead and a lower peak memory (11.97 GB vs 13.17 GB).

Why Ray was slower here. Ray processed the data in 4 passes and re-read the inputs each time (a memory-saving choice), ran an explicit hash shuffle for deduplication, and processed pandas batches in Python, whereas Spark optimized the whole job as one query. Ray's low average CPU (12.2%) suggests its 4 worker CPUs were often waiting rather than computing. A configuration with more memory or fewer passes might narrow the gap.

AI-first project. Ray is attractive when the pipeline sits next to model training or inference in Python, because custom Python code, GPUs and ML libraries run natively with small overhead (shown by the UDF result). This part is judgment based on the UDF result and the frameworks' design, not something these runs measured.

BI-first project. Spark is the better choice for large structured transformations, joins and aggregations: in this benchmark it was far faster on the SQL-style pipeline and needed no manual tuning of passes.

7. Limitations

- One run per framework; no repeats, so run-to-run variance is unknown. The time gap is large, but the exact ratio should be read as indicative.
- Both clusters run on one 12 GiB laptop, so there is no real network between workers, and CPU and memory are system-wide readings that include the OS, the Spark driver and the Ray head.
- The Ray implementation used a 4-pass design; the Spark implementation did not, so the comparison is of two tuned pipelines, not a pure engine-to-engine measurement. The row-by-row Spark UDF was not benchmarked on the full dataset.